

Dynatrace Monitoring System

Requirement:

1. Ec2 instance (Flipkart)
2. Dynatrace

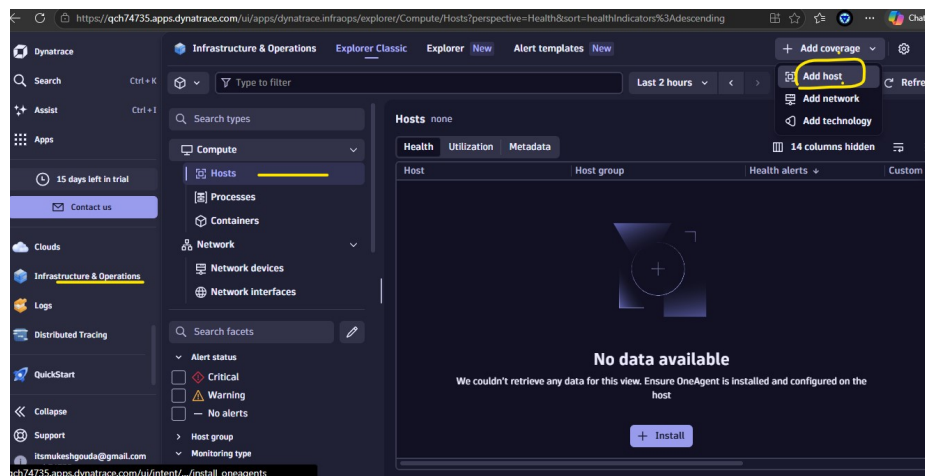
Tasks –

Install logs and observe

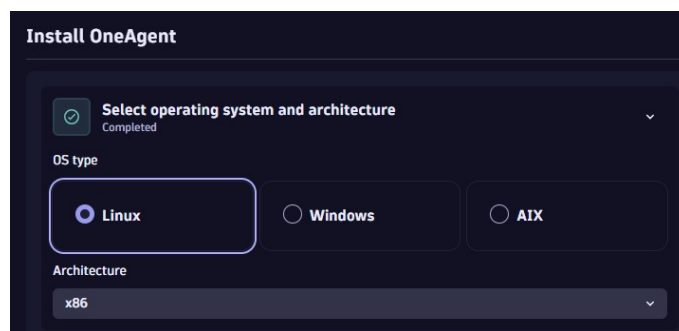
Add custom log

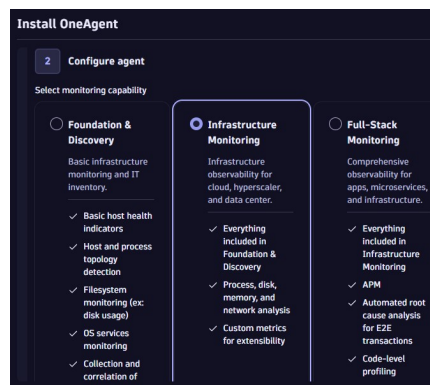
1. Create alert for cpu ram etc
2. Create dashboards
3. Create workflow with pagerduty
4. Create an ec2 instance (Flipkart) and login as root user .

Add host/server .

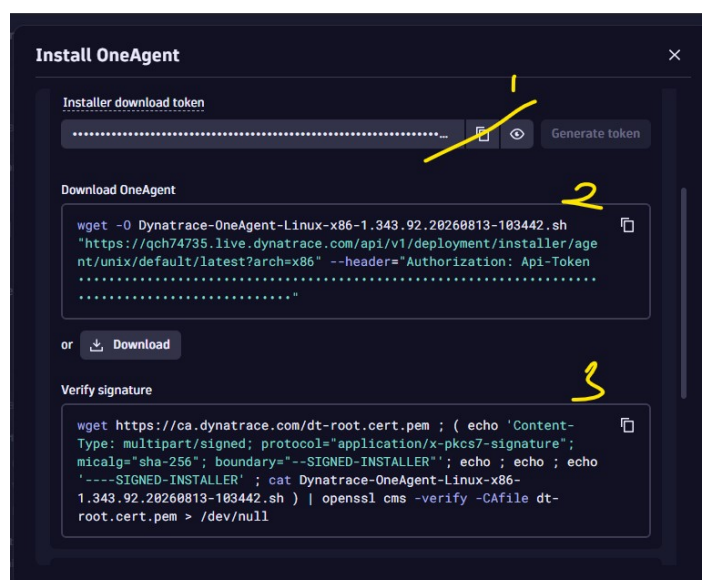


Select operating system and architecture : linux & x86

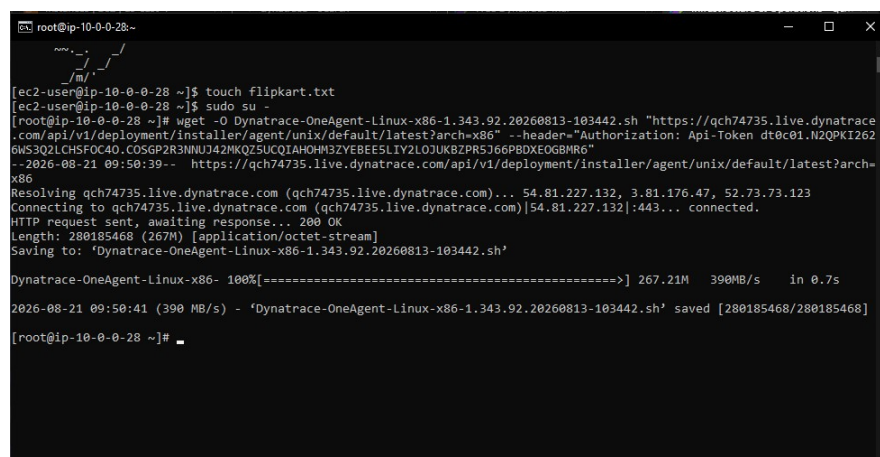




Generate key and install the agent downloader in ec2 and verify .



Install the agent



Verify

```
[root@ip-10-0-0-28 ~]# wget https://ca.dynatrace.com/dt-root.cert.pem ; ( echo 'Content-Type: multipart/signed; protocol
--application/x-pkcs7-signature'; micalg="sha-256"; boundary="--SIGNED-INSTALLER"; echo ; echo ; echo "--SIGNED-INSTA
LLER' ; cat Dynatrace-OneAgent-Linux-x86-1.343.92.20260813-103442.sh ) | openssl cms -verify -CAfile dt-root.cert.pem >
/dev/null
--2026-08-21 09:51:50-- https://ca.dynatrace.com/dt-root.cert.pem
Resolving ca.dynatrace.com (ca.dynatrace.com)... 18.165.83.54, 18.165.83.84, 18.165.83.58, ...
Connecting to ca.dynatrace.com (ca.dynatrace.com)|18.165.83.54|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 4340 (4.2K) [binary/octet-stream]
Saving to: 'dt-root.cert.pem'

dt-root.cert.pem      100%[=====] 4.24K --KB/s in 0s

2026-08-21 09:51:50 (115 MB/s) - 'dt-root.cert.pem' saved [4340/4340]

CMS Verification successful
[root@ip-10-0-0-28 ~]#
```

Now install the- OneAgent:

4

Install

Install OneAgent as the privileged user

```
/bin/sh Dynatrace-OneAgent-Linux-x86-1.343.92.20260813-103442.sh --
set-monitoring-mode=infra-only --set-app-log-content-access=true
```

Show deployment status

Step 4 of 4

Back

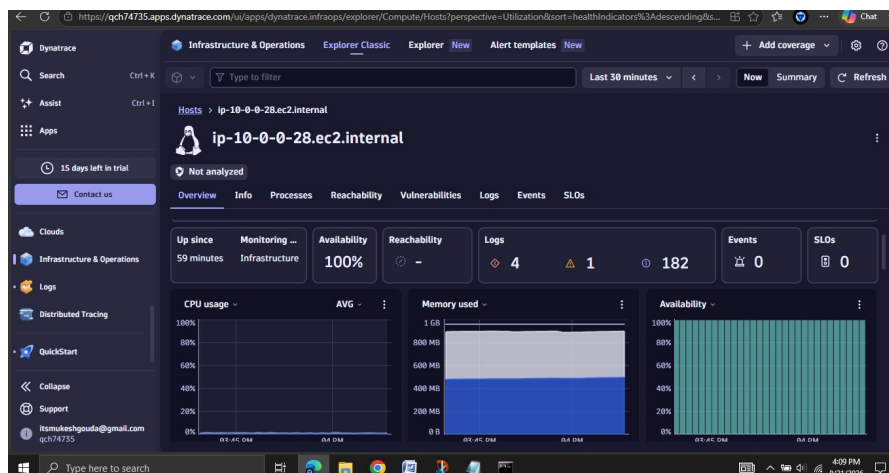
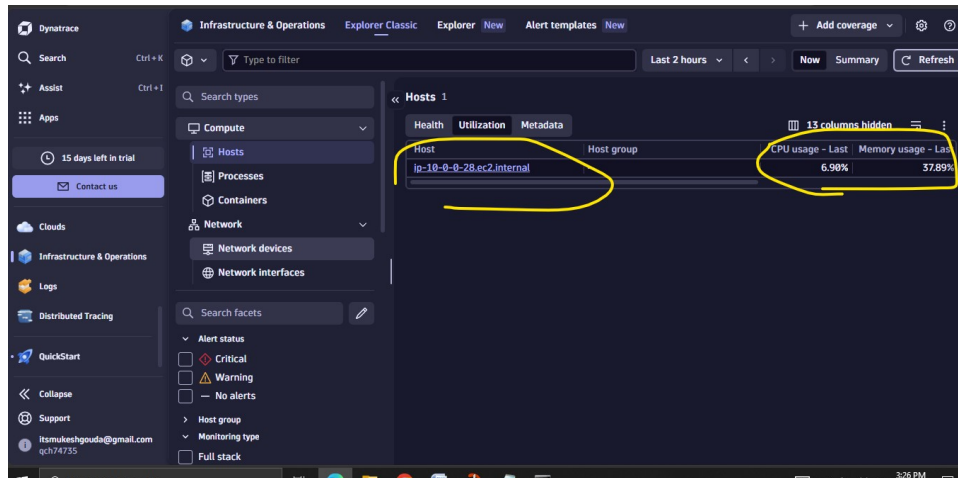
Close

```
root@ip-10-0-0-28 ~# /bin/sh Dynatrace-OneAgent-Linux-x86-1.343.92.20260813-103442.sh --set-monitoring-mode=infra-only --set-app-log-content-access=true
09:54:06 Checking root privileges...
09:54:06 Process real user: root, real ID: 0
09:54:06 Process effective user: root, effective ID: 0
09:54:06 Process root access: true
09:54:07 Logging to /var/log/dynatrace/oneagent/installer/installation.2564.log
09:54:07 Installation started, version 1.343.92.20260813-103442, build date: 13.08.2026, PID 2564.
09:54:07 Detected platform: LINUX arch: X86_64
09:54:07 Checking /opt/dynatrace/oneagent for prohibited mount flags
09:54:07 Checking free space in /opt/dynatrace/oneagent
09:54:07 Extracting...
09:54:08 Unpacking. This may take a while...
09:54:25 Unpacking complete.
09:54:25 Checking if an operating system is supported...
09:54:25 The operating system is supported, continuing installation.
09:54:26 Moving new binaries into lib folders...
09:54:26 User 'dtuser' added successfully.
09:54:32 Non-privileged mode is enabled.
09:54:32 Checking free space in /lib64
09:54:32 Storing SELinux policy sources in /opt/dynatrace/oneagent/agent.
09:54:34 Installing SELinux dynatrace_oneagent module, this may take a while...
09:54:44 dynatrace_oneagent module was successfully installed
09:54:44 SELinux dynatrace_oneagent module installation finished.
09:54:44 Verifying auto-injection compatibility...
09:54:44 Auto-injection compatibility check result: OK
09:54:44 Refreshing dynamic linker runtime bindings using ldconfig
09:54:45 Updating /proc/sys/kernel/core_pattern with oneagentdumpproc
09:54:59 Starting agents...
09:54:59 oneagent service started
09:54:59 Checking if agent is connected to the server...
09:55:01 Dynatrace OneAgent has successfully connected to Dynatrace Cluster Node. You are all set. The Dynatrace OneAgent is fully operational.
[root@ip-10-0-0-28 ~]#
```

#systemctl start oneagent

systemctl status oneagent

Now refresh the page and check the metrics



Now increase the load using stress command .

#yum install stress-y -> install the stress package

stress -version

Stress the CPU: To stress 4 CPU cores for 10 seconds:

#stress -c 4 --timeout 10

Stress Memory: To stress memory with 2 workers, each allocating 128MB, for 10 seconds:

stress --vm 2 --vm-bytes 128M --timeout 10

Combine CPU and Memory Stress:

```
stress -c 4 --vm 2 --vm-bytes 128M --timeout 5
```

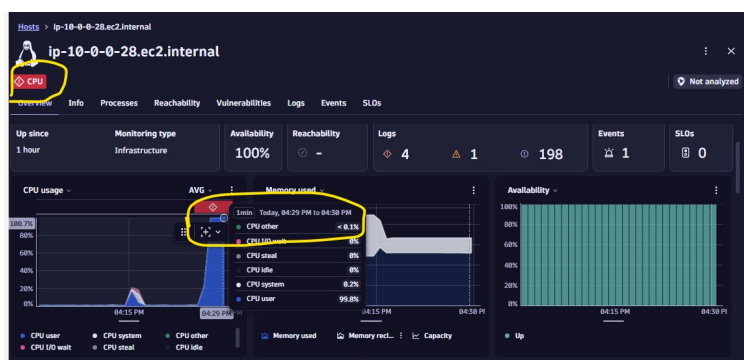
Stress Disk: To stress disk with 2 workers writing 1GB each:

```
stress --hdd 2 --hdd-bytes 1G --timeout 10
```

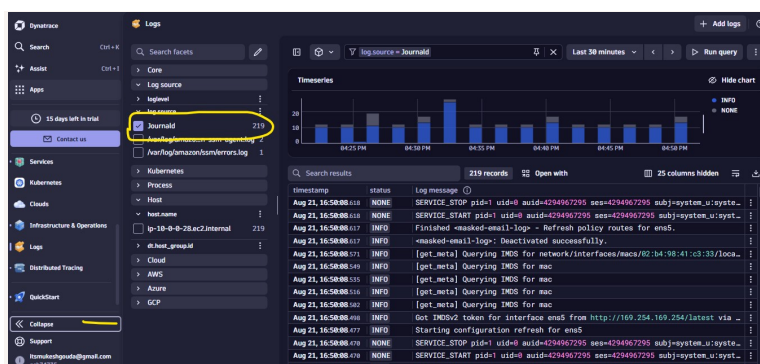
Now increase the stress for 300 s

```
#stress -c 4 --timeout 300
```

```
[root@ip-10-0-0-28 ~]# stress -c 4 --timeout 300
stress: info: [33773] dispatching hogs: 4 cpu, 0 io, 0 vm, 0 hdd
```



You can check logs of the instance

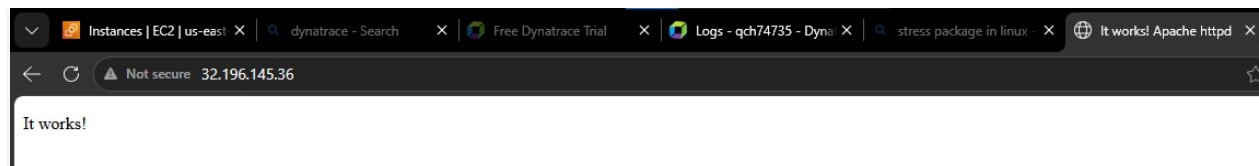


Now try to install httpd service and observe if the logs stored or not .

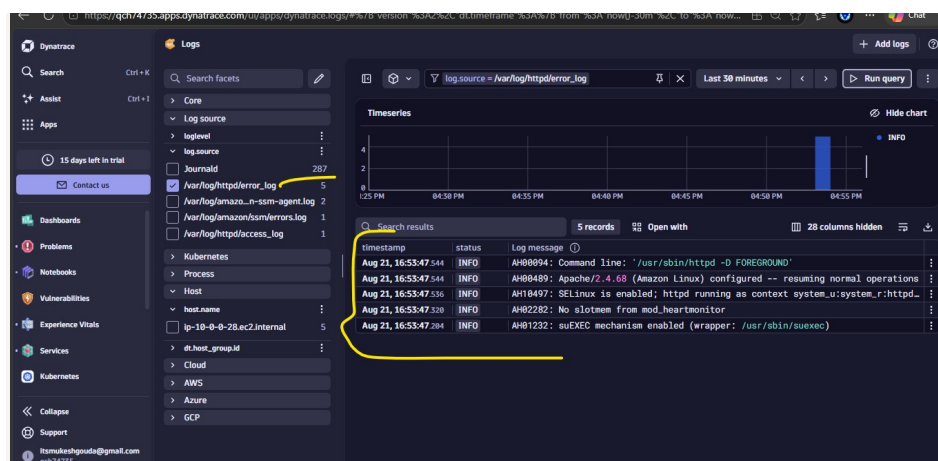
```
#yum install httpd -y
```

```
#systemctl start httpd
```

```
[root@ip-10-0-0-28 ~]# systemctl start httpd
[root@ip-10-0-0-28 ~]# ss -tlnl
Netid      State      Recv-Q     Send-Q      Local Address:Port      Peer Address:Port
udp        UNCONN     0           0            127.0.0.1:323           0.0.0.0:*
udp        UNCONN     0           0            10.0.0.28%ens5:68       0.0.0.0:*
udp        UNCONN     0           0            [::1]:323              [::]:*
udp        UNCONN     0           0            [fe80::b4:98ff:fe41:c333]%ens5:546 [::]:*
tcp        LISTEN     0           128          0.0.0.0:22             0.0.0.0:*
tcp        LISTEN     0           128          127.0.0.1:14499        0.0.0.0:*
tcp        LISTEN     0           511          *:80                   *:.*
tcp        LISTEN     0           128          [::]:22                [::]:*
```



One agent automatically detect the log file and show the logs

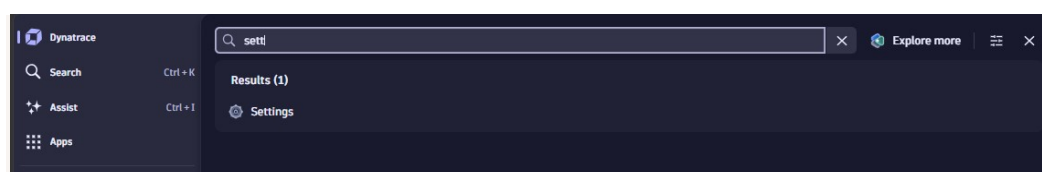


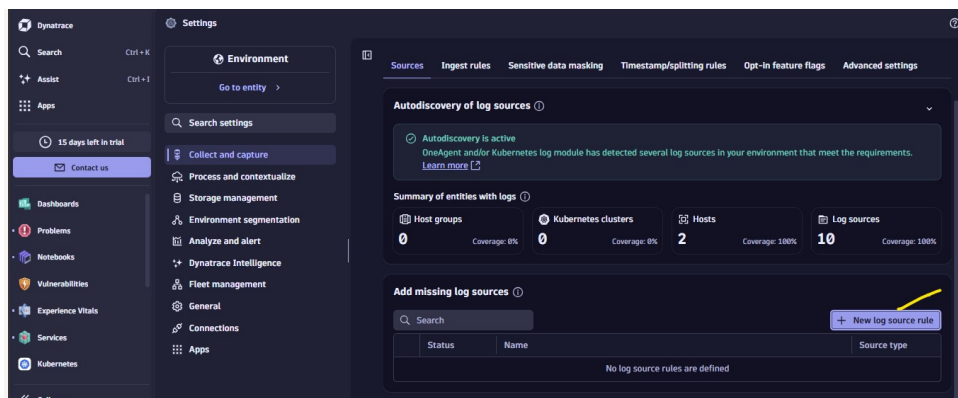
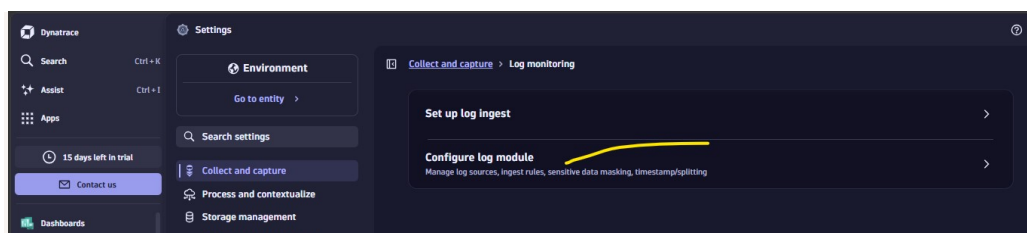
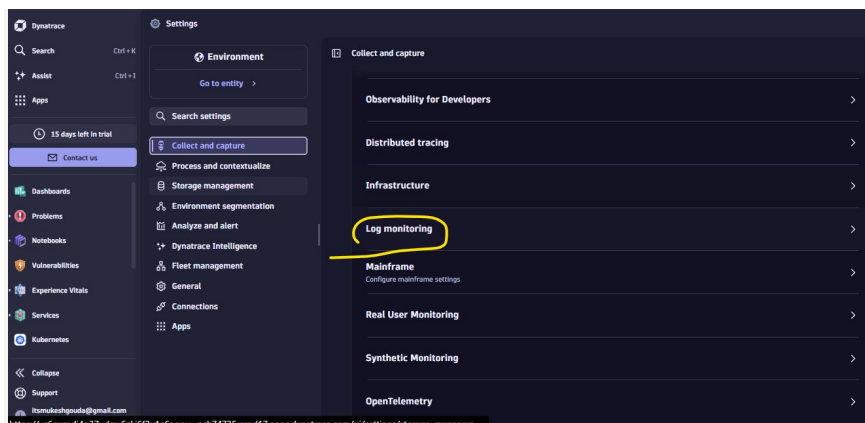
We can also configure custom log file to track.

Create a .log file and add the path in the dynatrace

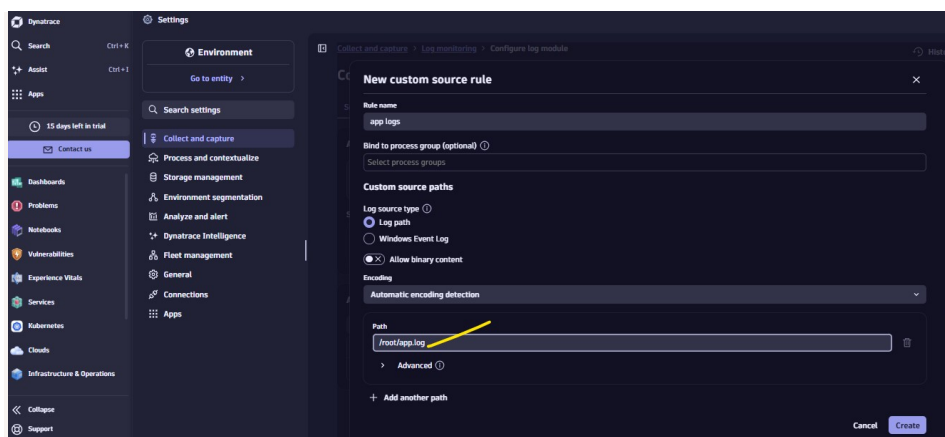
```
[root@ip-10-0-0-28 ~]# ls
Dynatrace-OneAgent-Linux-x86-1.343.92.20260813-103442.sh dt-root.cert.pem
[root@ip-10-0-0-28 ~]# touch app.log
[root@ip-10-0-0-28 ~]# ls
Dynatrace-OneAgent-Linux-x86-1.343.92.20260813-103442.sh app.log dt-root.cert.pem
[root@ip-10-0-0-28 ~]#
```

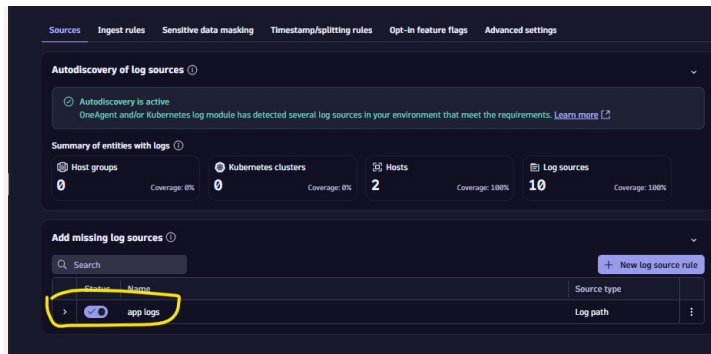
Search settings





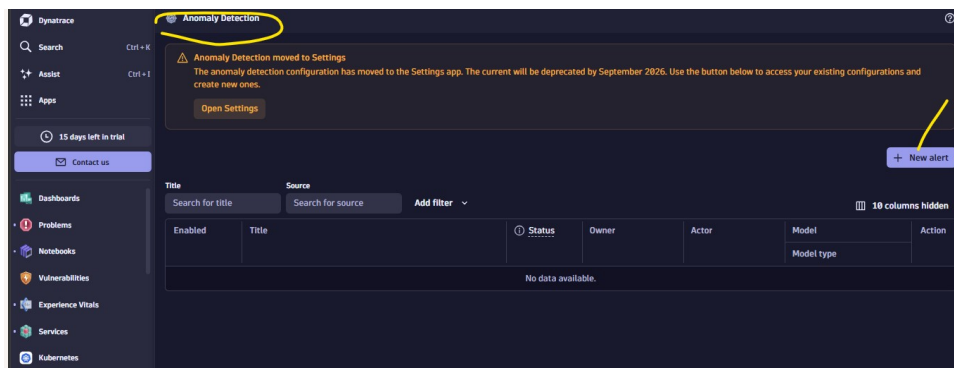
Give log folder and and path



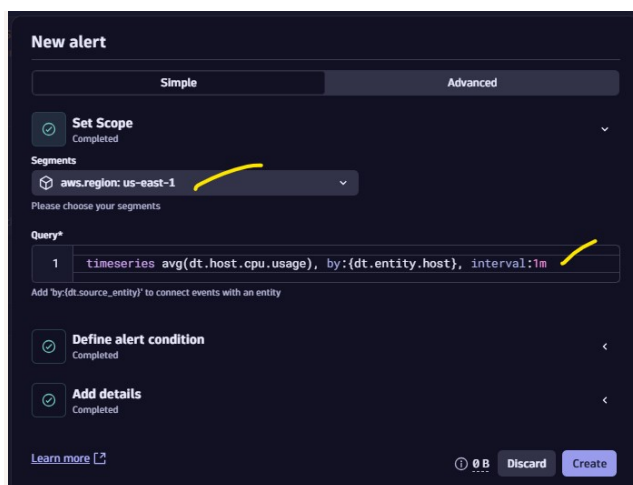


Create alert for server cpu

Go to search > Anomaly Detection > create alert



Set alert details



New alert

Define alert condition Completed

Select use case

- ☒ Static threshold
For metrics that do not change over time
- ☐ Auto adaptive threshold
For metrics that change over time
- ☐ Seasonal Baseline
For metrics that show seasonal behavior

Set a condition

Click "Suggest values" to assist you in defining the appropriate static threshold.

Threshold* Unit Undefined Suggest values

Alert condition Alert if metric is above

Click preview to see a graphic representation of your alert Preview

Learn more Discard Create

New alert

Simple Advanced

Set Scope Completed

Define alert condition Completed

Add details Completed

Title* CPU usage spike

Provide a title for your Custom Alert to facilitate future searches.

Event name* Devops_CPU

If the alarm condition is violated, an alarm event is created and triggering, for example, Davis problems for further analysis. Give this alert event a defined name, for example: CPU high alert on entry {dt.source_entity}

Learn more Discard Create

Wait till status shows **success**

Anomaly Detection

Anomaly Detection moved to Settings
The anomaly detection configuration has moved to the Settings app. The current will be deprecated by September 2026. Use the button below to access your existing configurations and create new ones.

Open Settings

15 days left in trial Contact us + New alert

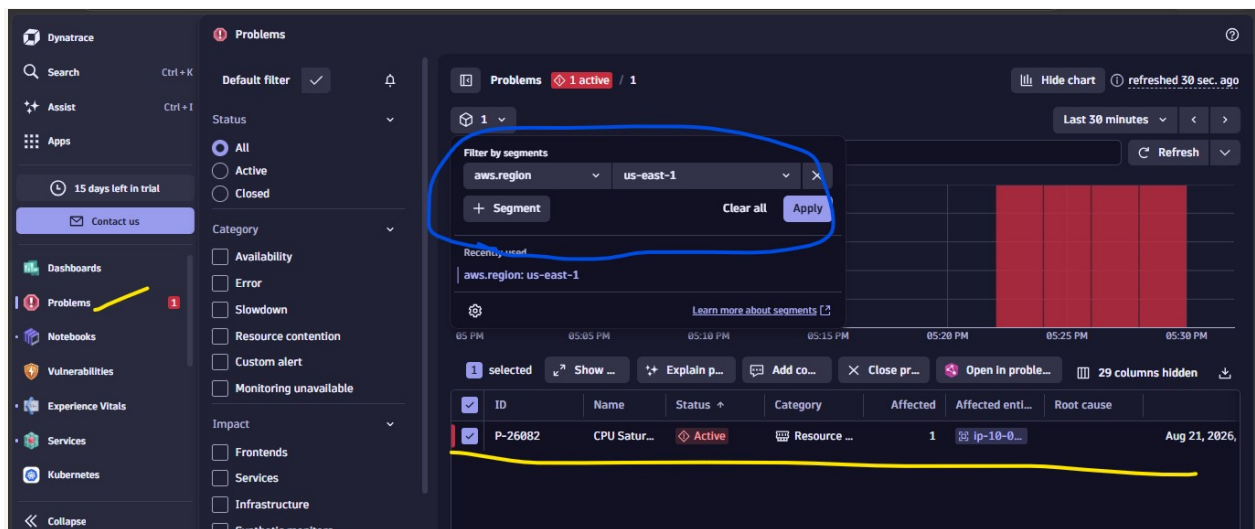
Enabled	Title	Source	Status	Owner	Actor	Model	Model type	Action
☒	CPU usage spike		Success	ismukeshgouda@...	ismukeshgouda@...	STATIC_THRESHOLD		

Now login to ec2 and intentionally increase the cpu load

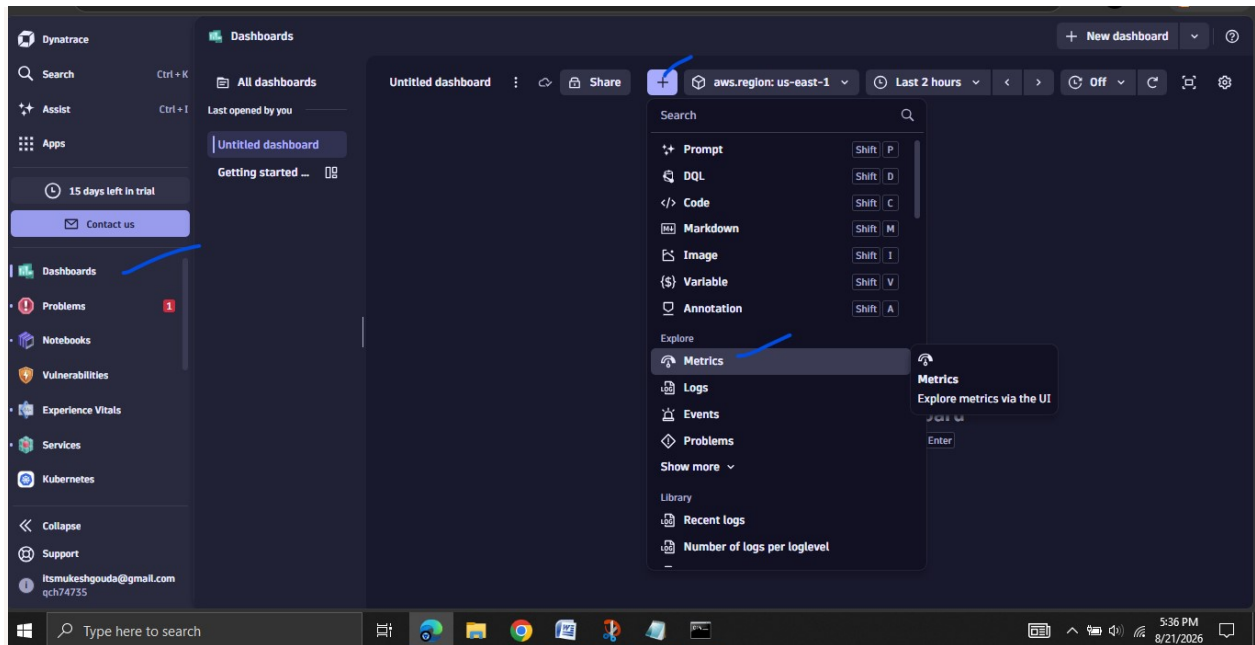
```

[root@ip-10-0-0-28 ~]# stress -c 4 --timeout 300
stress: info: [39482] dispatching hogs: 4 cpu, 0 io, 0 vm, 0 hdd
    
```

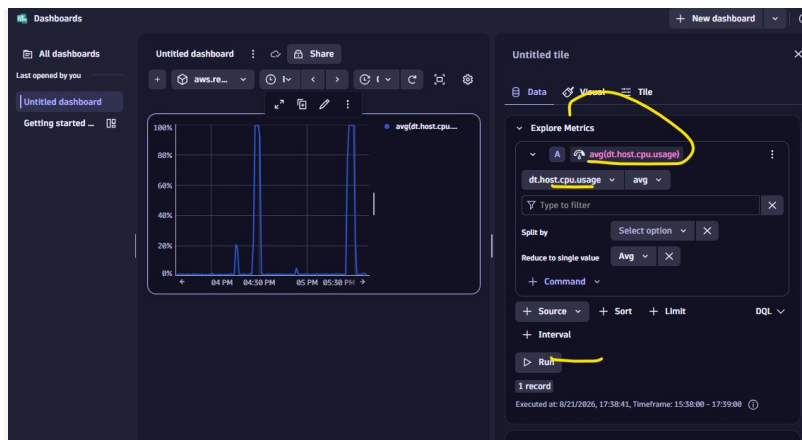
Now go to problem tab and filter active alerts . **[Make sure you are selecting the host deployed zone]**



We can also make our custom dashboard like grafana :



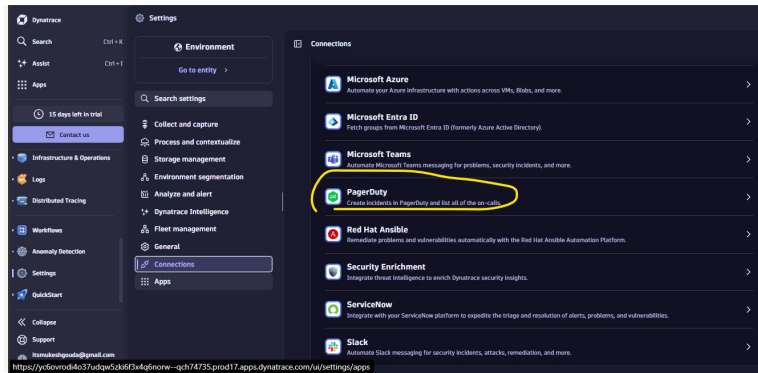
Create alert for cpu



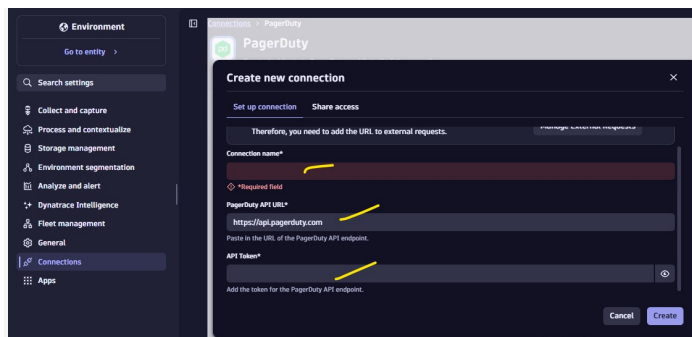
Dynatrace workflow for Pagerduty

We can make a workflow so that the pager duty will trigger an event when any event or incident create

First create a connection



By giving url and token it will create a connection



Later in workflow you can give connection name and event information it will trigger an incident.

